ResumeLens

Mateo Cuenca - Jazmin Michelle Galvez - Gabriela Cardona
Computación y Estructuras Discretas III - Universidad Icesi 2026-2

Research question

Can regular expressions, finite-state transducers, finite automata, and context-free grammars work together to check — not rank — whether a resume matches a formal pattern for a job profile?

STAGE 1
Information Extraction
Regular Expressions

→

STAGE 2
Normalization
Finite-State Transducers

→

STAGE 3
Recognition
Finite Automata

→

STAGE 4
Candidate Profile
Context-Free Grammar

1 - Introduction

- Many resume-screening tools use machine learning to rank candidates.
- ResumeLens does something different: it only checks if the skills in a resume match a formal pattern.
- Each model has one job. Rules extract simple fields like emails and degrees [1][2]. Transducers standardize spellings. Automata check the pattern. A grammar organizes the profile, with optional and repeated fields.
- ResumeLens uses four job profiles: Full Stack Developer and Machine Learning Engineer (required by the assignment), plus DevOps Engineer and Data Engineer (our choice).

2 - Literature

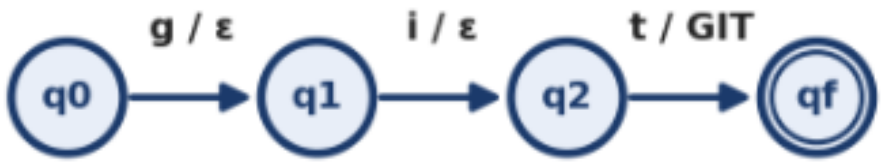
- Resume parsing with rules and NLP [1][2]
- Transducers standardize text variants [4][5]
- Automata find multi-word patterns [3]
- Grammars describe CV languages [6][7]
- Pyformlang is a library for all these models [8]

3 - Methods

- **Stage 1 – Regular expressions**. Python regular expressions find contact data, skills, education, and experience. One shared rule checks known terms from the most specific to the most general. It skips a match right after a dot, so “react.js” is not also read as “JS”.
- **Stage 2 – Transducers**. A finite-state transducer changes each spelling into one standard word. For example, “JS” and “Javascript” both become JAVASCRIPT. It is defined by 7 parts: (Q, Σ, Γ, δ, ω, q0, F). Then the words are sorted in the order of the job profile.
- **Stage 3 – Automata**. A deterministic finite automaton (DFA) checks the sorted words. It is defined by 5 parts: (Q, Σ, δ, q0, F). Each state counts one more required category. The resume is accepted only if every category appears, in order.
- **Stage 4 – Grammar**. A textX grammar defines how the candidate profile is written. Three extra rules check what a grammar cannot: each profile appears once, each word belongs to its profile, and each verdict matches the automaton. A valid profile becomes an HTML or Markdown page.

Stage 2 - Figure 2

Figure 2. Transducer for GIT.
It reads one character at a time, and only the last one prints the word.



Stage 3 - Figure 1

Figure 1. DFA of the Machine Learning Engineer profile.
Each state means one more required category was found. Only q7 accepts.



ResumeLens

Mateo Cuenca - Jazmin Michelle Galvez - Gabriela Cardona
Computación y Estructuras Discretas III - Universidad Icesi 2026-2

4 - Results

Example: Wednesday Addams

1. **Raw skills**:
JS, React.js, NodeJS, Postgres, REST APIs, Git

2. **Normalized**:
JAVASCRIPT, REACT, NODE_JS, REST_API, POSTGRESQL, GIT

3. **Verdict**:
Full Stack Developer ACCEPTED. The other three profiles: REJECTED.

4. **Output**:
The validated profile is shown as an HTML page.

69/69

automated tests passing

Results (continued)

- The tests check each stage alone. They also run four full sample resumes, one per profile, through the whole pipeline.
- On these four resumes, each one is accepted only by its own profile.
- We found and fixed one real bug. The rule that stops “JS” from matching inside longer words also treated a dot as a word boundary, so “react.js” was also read as “JS”. A test now covers it.
- Stage 4 rejects bad profiles, such as a wrong verdict or a word that does not belong to the profile.

5- Conclusions

- Four formal models work together as one pipeline that we can test.
- Every accept or reject decision can be traced to one path in an automaton. There is no hidden model.
- Limit: ResumeLens only knows the words in its Stage 2 tables. An unknown word or a typo is marked as unknown, not guessed.

References

[1] Rule-Based Information Extraction from Multi-format Resumes for Automated Candidate Screening. IIETA.

[2] Resume Parser with Natural Language Processing. IRJET.

[3] Finite State Automata on Multi-Word Units for Efficient Text-Mining. Mathematics (MDPI), 12(4), 506.

[4] Finite-State Transducers in Language and Speech Processing. Computational Linguistics, 23(2).

[5] Weighted Finite-State Transducers for Normalization of Historical Texts. Natural Language Engineering, Cambridge University Press.

[6] Synthesis of CVs Using a Context-free Grammar. ASTESJ, 6(2), 122-130.

[7] Inferring Context-Free Grammars for Domain-Specific Languages. Electronic Notes in Theoretical Computer Science.

[8] Pyformlang: An Educational Library for Formal Language Manipulation.